Smart City Complaint & Service Portal

FULL-STACK PROJECT REPORT

Backend:	Spring Boot with JWT Authentication
Frontend:	React.js (with Vite & Shadcn UI)
Database:	PostgreSQL Database
Prepared For:	Project Submission
Date:	July 2026

1. Introduction

The Smart City Complaint & Service Portal is an advanced full-stack web application designed to bridge the communication gap between municipal citizens and local government authorities. In modern urban environments, registering infrastructure or utility faults often involves complex, non-transparent bureaucratic processes. This platform digitizes and streamlines that workflow.

The system allows citizens to securely report localized public issues—such as road defects, sanitation breakdown, electricity outages, and general municipal services. Departmental officers are then systematically assigned to these issues, allowing them to manage progress and seamlessly update resolution states. Concurrently, system administrators retain high-level overview access to manage users, add departmental categories, and evaluate performance benchmarks to secure operational excellence.

2. System Architecture

The platform is designed around a strictly separated, decoupled full-stack architecture running a Java-based backend ecosystem and a JavaScript single-page application frontend. Communication is managed entirely via an asynchronous RESTful API tier.

Architecture Overview

```
User Browser
    ↓
React Frontend
    ↓ [Axios REST API]
Spring Boot Backend with JWT Authentication
    ↓
Controller Layer → Service Layer → Repository Layer
    (Supporting Architecture: DTOs, Enums, Custom Exceptions)
    ↓ [Spring Data JPA]
PostgreSQL Database
```

Backend Architecture Layering

- **Controller Layer:** Intercepts incoming HTTP requests, maps specific REST endpoints, parses payload objects, and offloads processing tasks to the core domain. Examples include `UserController`, `AuthController`, and `ComplaintController`.
- **DTO (Data Transfer Object) Layer:** Decouples internal persistent database entities from external API request payloads and response structures, enforcing clear communication boundaries and payload data validation.
- **Service Layer:** Encapsulates granular business rule executions, transaction logic boundaries, context evaluations, and authorization routing checks.
- **Repository Layer:** Leverages Spring Data JPA abstraction to interface seamlessly with the persistent relational PostgreSQL engine without managing raw native SQL overhead manually.
- **Enum Layer:** Strongly types immutable system configurations, managing defined roles (`CITIZEN`, `OFFICER`, `ADMIN`) and tracking state configurations (`PENDING`, `IN_PROGRESS`, `RESOLVED`) across application layers.
- **Exception Handling Layer:** Deploys specialized custom domain exceptions (e.g., `ResourceNotFoundException`, `InvalidCredentialsException`) managed via global controller advice structures to produce clean, uniform JSON error responses.
- **Security Layer (Spring Boot Backend with JWT Authentication):** Enforces mandatory authentication filters using Spring Security, stateful verification processing via JSON Web Tokens (JWT), role-level route configurations, and cryptographically safe BCrypt password hashing.

Authentication Protocol

Upon verification, requests must attach a cryptographically signed header:

```
Authorization: Bearer <JWT_TOKEN>
```

3. Main Features

Role-Based Access Control

The application provisions tailored access interfaces based on strict access control criteria split among three global roles:

- **CITIZEN:** Public account layer focusing on creation, categorization, tracking, and auditing of personalized complaints.
- **OFFICER:** Operationally constrained profiles dedicated to assessing issues allocated explicitly to their associated department.
- **ADMIN:** Unrestricted super-user layer handling foundational configurations, platform metadata updates, analytics monitoring, and user records.

Workflow Progression Model

```
Citizen Account → Submit Complaint → Auto-Assignment → Officer Review → Status Update
```

Advanced Management & Metrics

Administrators have access to a visual metric framework focused on Service Level Agreements (SLAs). Built-in tracking dashboard metrics evaluate status distributions and department efficiency profiles to map urban improvement benchmarks.

- **SLA Complaint Overdue:** Automated tracking flags that highlight high-priority complaints remaining unaddressed past established department deadlines.

4. Database Design

Data entity normalization is mapped natively using PostgreSQL to represent system configurations and relational constraints clearly.

Table Name	Core Fields / Attributes	Structural Relations
Users	id, full_name, email, password, role	One-to-Many with Complaints
Departments	id, name, description	One-to-Many with Complaints
Categories	id, name (e.g., Roads, Utilities, Sanitation)	One-to-Many with Complaints
Complaints	id, title, description, location, status, priority, created_at	Belongs to User, Dept, Category

5. Frontend Implementation

The single-page client interface is built with React.js using modular state paradigms and component-driven architecture. Highly functional interface primitives are deployed via Shadcn UI components backed by Tailwind CSS utilities. Application routes are guarded conditionally based on active identity contexts.

Project File Structure Matrix

The client application directory tree is structured systematically to detach structural visual pages, layouts, custom hooks, and shared endpoint abstractions:

```
frontend/
├── src/
│   ├── api/
│   ├── assets/
│   ├── components/
│   │   ├── dashboard/
│   │   ├── home/
│   │   ├── ui/
│   │   ├── AuthContext.jsx
│   │   ├── AuthProvider.jsx
│   │   ├── Footer.jsx
│   │   ├── mode-toggle.jsx
│   │   ├── Navigation.jsx
│   │   ├── ProtectedRoute.jsx
│   │   ├── PublicOnlyRoute.jsx
│   │   ├── theme-provider.jsx
│   │   └── useAuth.js
│   ├── lib/
│   ├── pages/
│   │   ├── dashboard/
│   │   ├── About.jsx
│   │   ├── Contact.jsx
│   │   ├── Home.jsx
│   │   ├── Login.jsx
│   │   └── Register.jsx
│   ├── App.css
│   ├── App.jsx
│   └── index.css
```

6. Challenges and Solutions

- **JWT Stateful Expiration Management:** Intermittent route failures occurred when local storage tokens expired during active browser sessions.
  Solution: Implemented global Axios interceptors to trap outgoing requests, checking status signatures and redirecting expired states gracefully.

- **Entity Mapping Loops:** Infinite recursion bugs were encountered during JSON normalization of bidirectional JPA associations (`@ManyToOne` and `@OneToMany`).
  Solution: Configured precise reference limits using Jackson annotations (`@JsonManagedReference` and `@JsonBackReference`).

7. Conclusion

In conclusion, the Smart City Complaint & Service Portal provides a secure, efficient way for citizens to report public issues and for municipal departments to resolve them quickly. By separating the React frontend from the Spring Boot backend, the platform ensures fast performance, smooth navigation, and strict security across different user roles. The system successfully automates manual paperwork and highlights critical, overdue complaints so they can be addressed without delay.